Pragya Chourasia

GitHub repository: https://github.com/pragyachourasia-05/data260-6758

DATA-260 Homework 8

Value	Definition	Mine
SID4	Last four digits of SJSU ID	6758
PORT_BASE	8000 + (SID4 mod 900)	8458
PREFIX	"s" + SID4	s6758
SEED	SID4	6758
VERIFY_SEED	260000 + SID4	266758
DOMAIN_ID	SID4 mod 8	6 - rental housing listings
Hardware		Intel Core i5-1335U, 16 GB RAM, Windows 11 (CPU inference)
Model		llama3.2:3b via Ollama (Part 2 agents)
Tagged commit	Tag hw8	44232fc167c8959c5879f507f49a780c5068d269

Part 1 - Listing Service over Kafka

My primary entity is Listing, so the operations are CREATE_LISTING and GET_LISTING. The client calls a FastAPI wrapper, the wrapper sends the request to Kafka, and a separate worker process stores listings in a Python dictionary and replies through Kafka.

1.1 Topics and message schemas

Topic	Used for
s6758_listing_requests	FastAPI sends requests, worker reads them
s6758_listing_responses	Worker sends responses, FastAPI reads them

Message	Schema
Request	{ "correlation_id": "unique-id", "reply_to": "s6758_listing_responses", "operation": "CREATE_LISTING", "data": { "property_address", "monthly_rent", "property_category", "listing_description" } }
CREATE response	{ "correlation_id": "...", "success": true, "listing_id": "generated-uuid", "message": "Listing created" }
GET request data	{ "listing_id": "existing-listing-id" }
GET response	{ "correlation_id": "...", "success": true, "listing": { "listing_id": "...", ...listing fields... } }
Error response	{ "correlation_id": "...", "success": false, "error": "Listing not found" }

1.2 correlation_id flow

Step	What happens to correlation_id
1. Client	homework_demo.py creates it once with uuid4 and sends it to FastAPI
2. FastAPI	Forwards it unchanged in the request and keeps a pending future under that id
3. Worker	Reads it from s6758_listing_requests and copies it into the response, also for errors
4. FastAPI	Reads s6758_listing_responses and uses the id to find the waiting request
5. Client	Receives the response and checks the id is the same one it sent

1.3 Docker containers

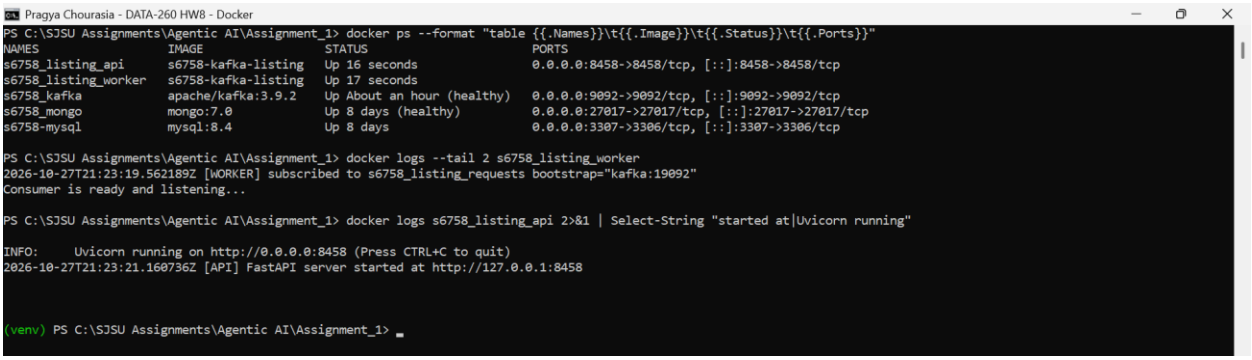
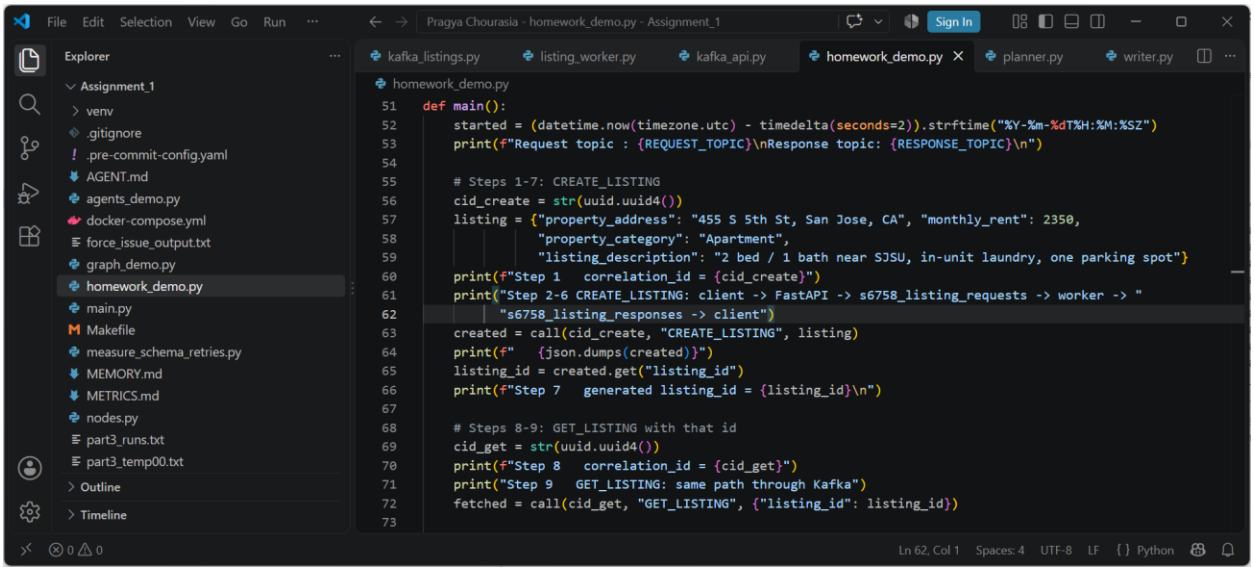


Figure 1.1: docker ps with Kafka, worker and FastAPI running, plus worker and FastAPI startup logs

1.4 homework_demo.py

homework_demo.py (homework_demo.py)



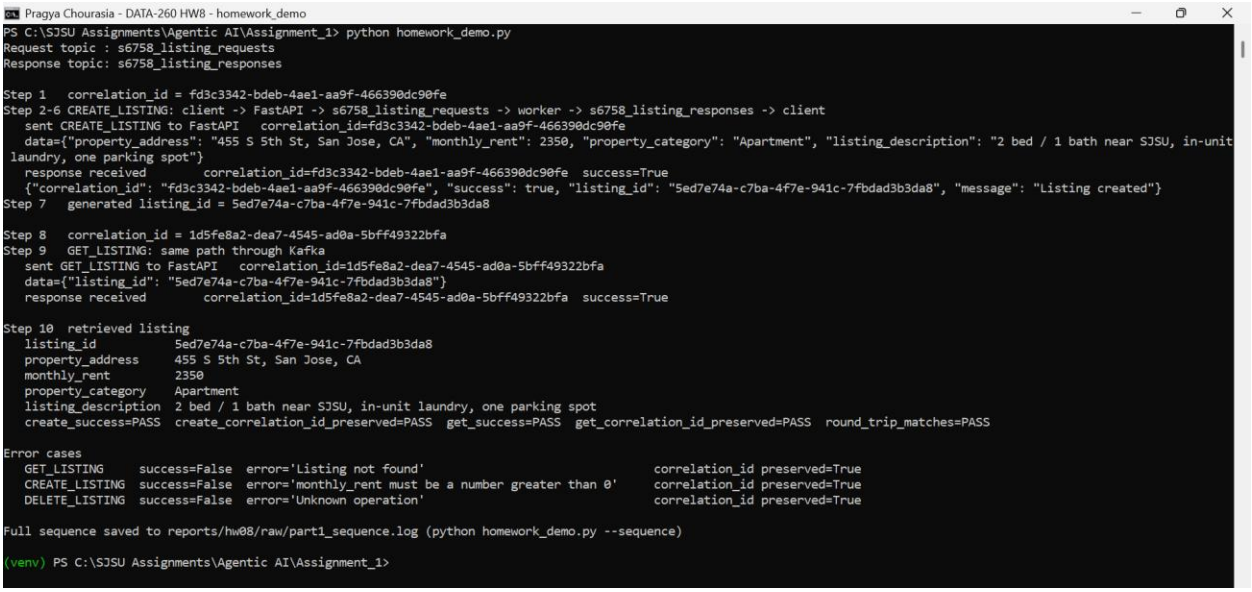
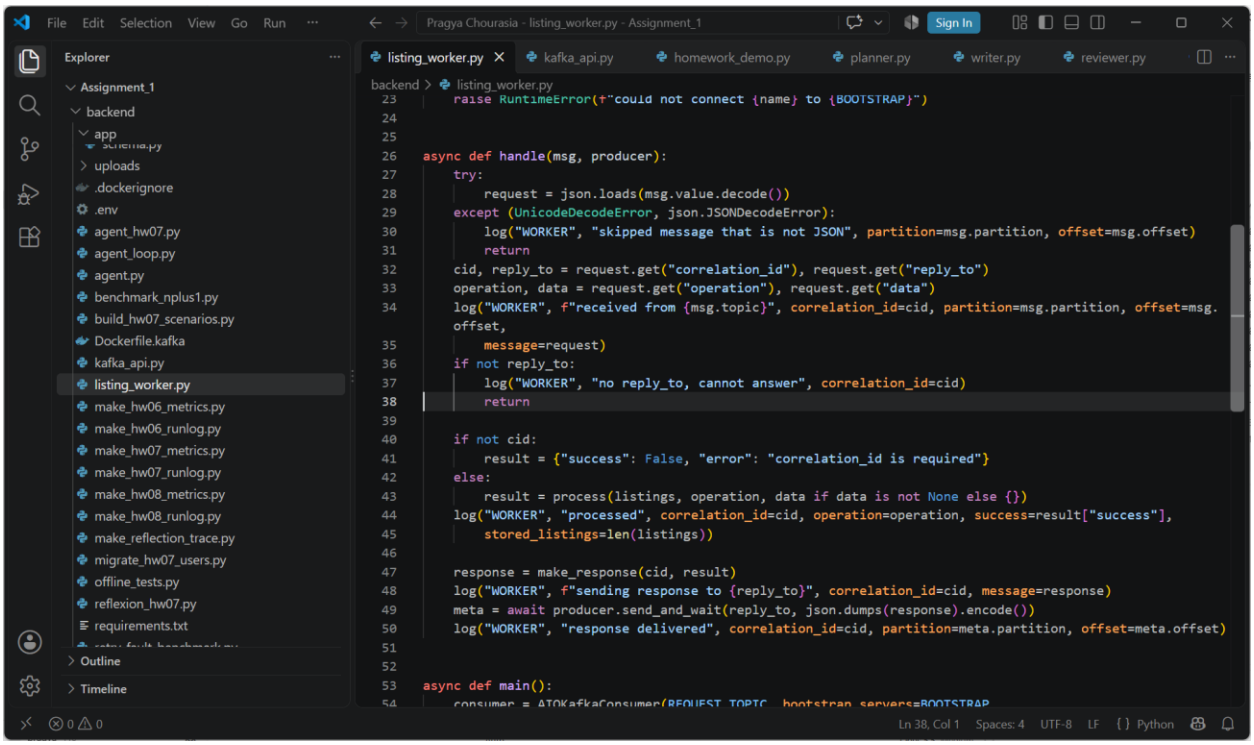


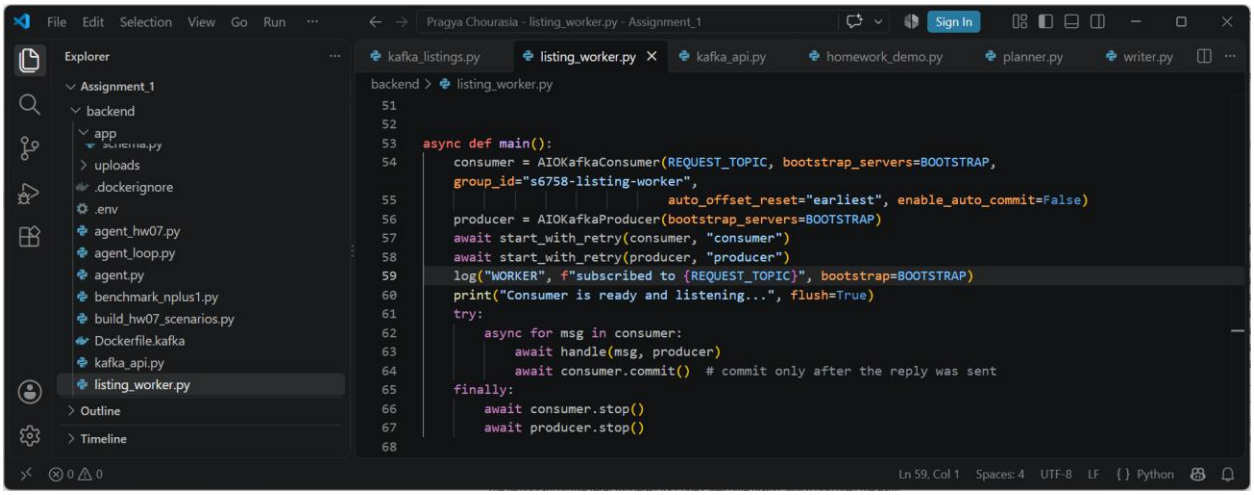
Figure 1.2: homework_demo.py: CREATE_LISTING, GET_LISTING and error cases

1.5 Worker

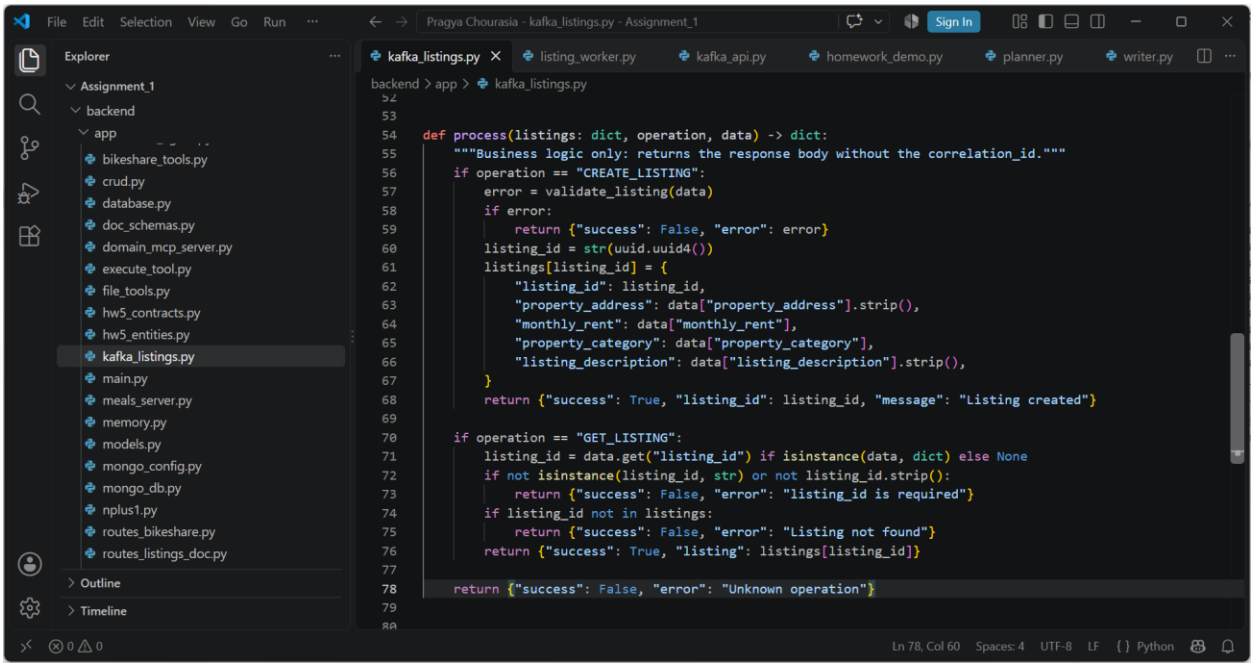
Worker: read, process, reply (backend/listing_worker.py)



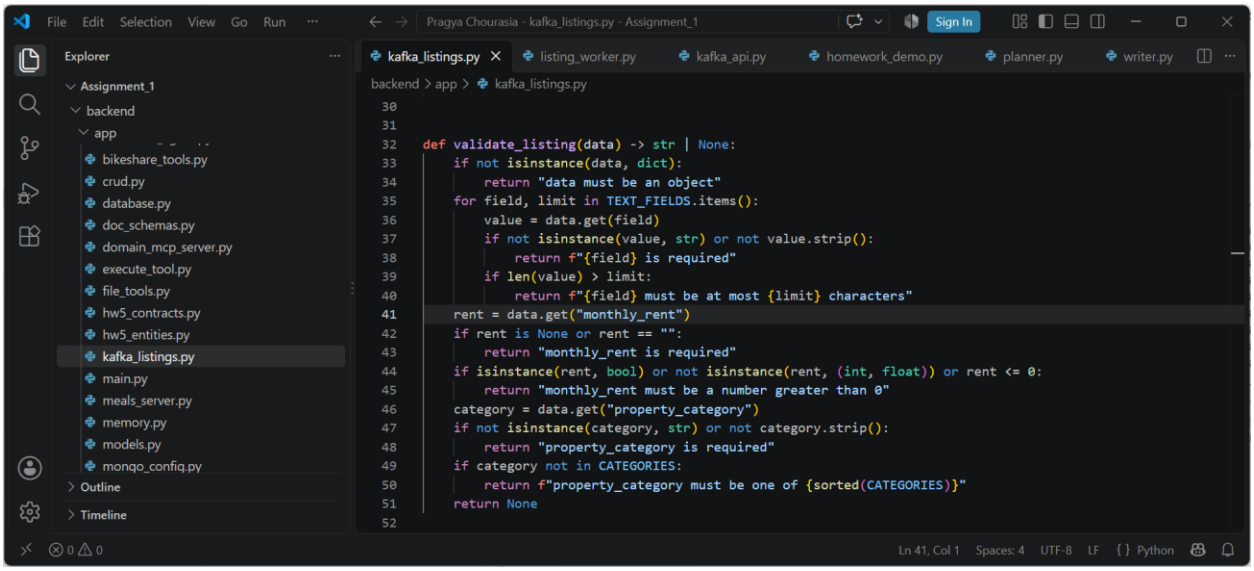
Worker: consumer loop (backend/listing_worker.py)



CREATE_LISTING and GET_LISTING (backend/app/kafka_listings.py)



Field validation (backend/app/kafka_listings.py)



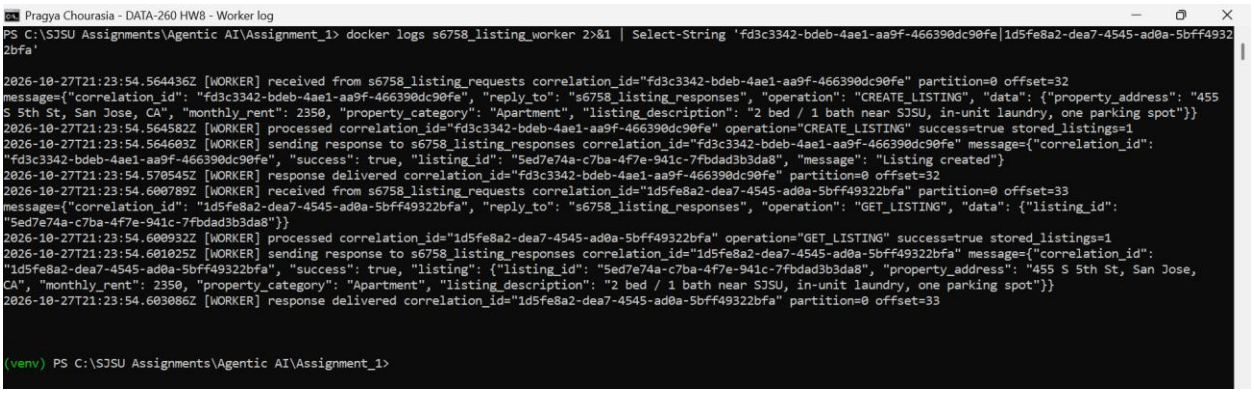
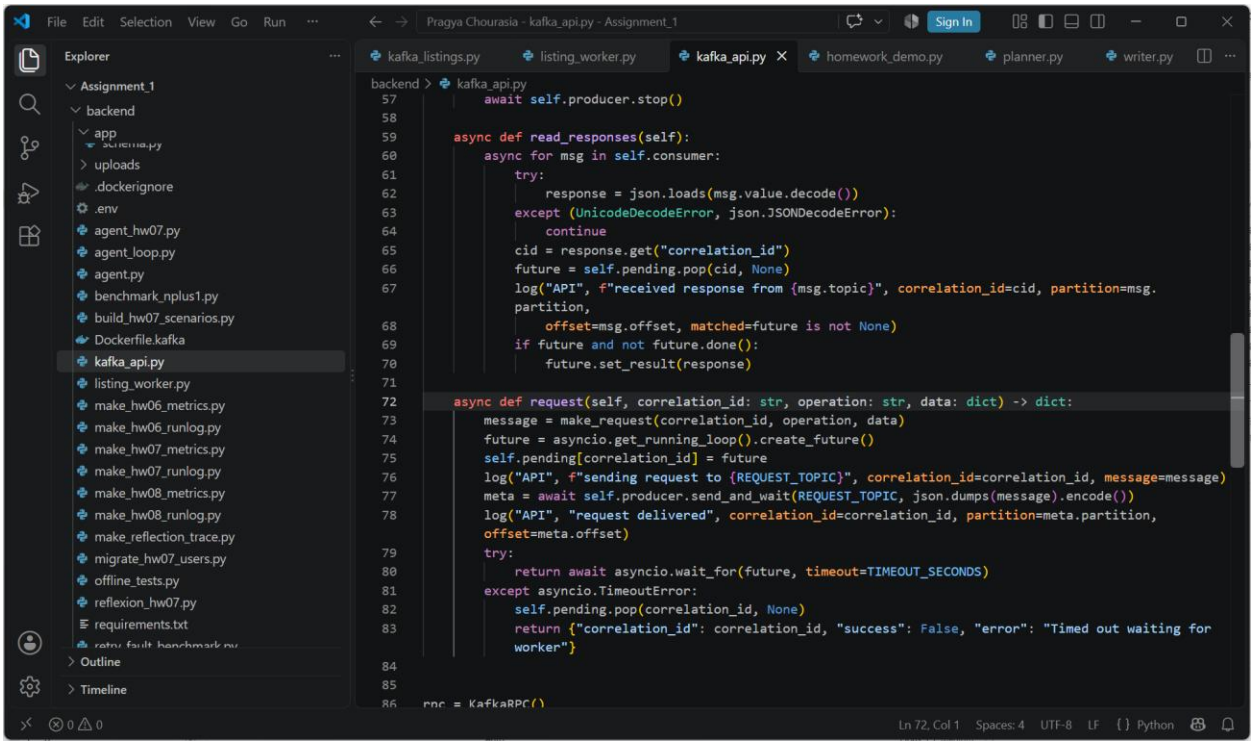


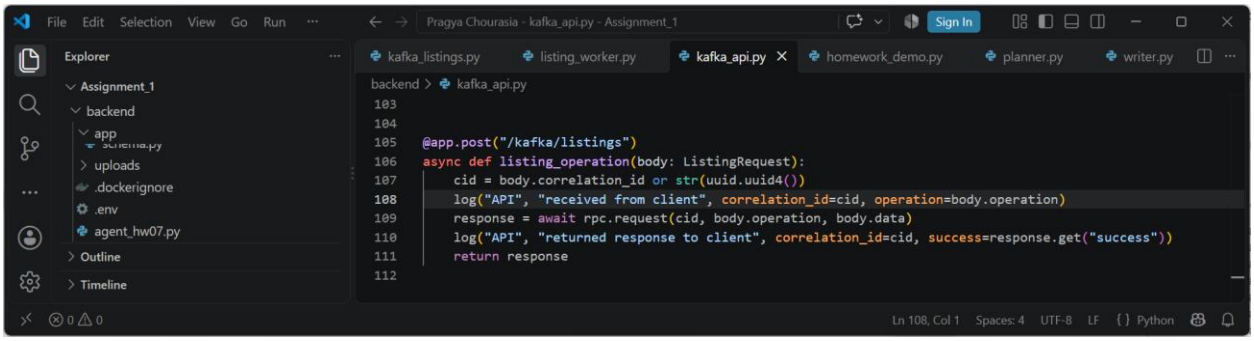
Figure 1.3a: Worker log: received, processed and response sent

1.6 Kafka client and API wrapper

Kafka client in the API wrapper (backend/kafka_api.py)



API wrapper route (backend/kafka_api.py)



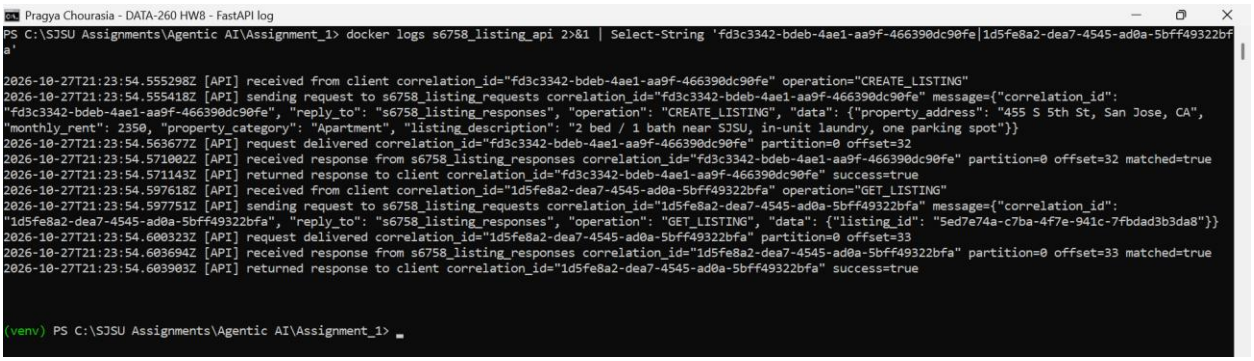


Figure 1.3b: FastAPI log: request sent to Kafka, response received and returned

1.7 Full sequence

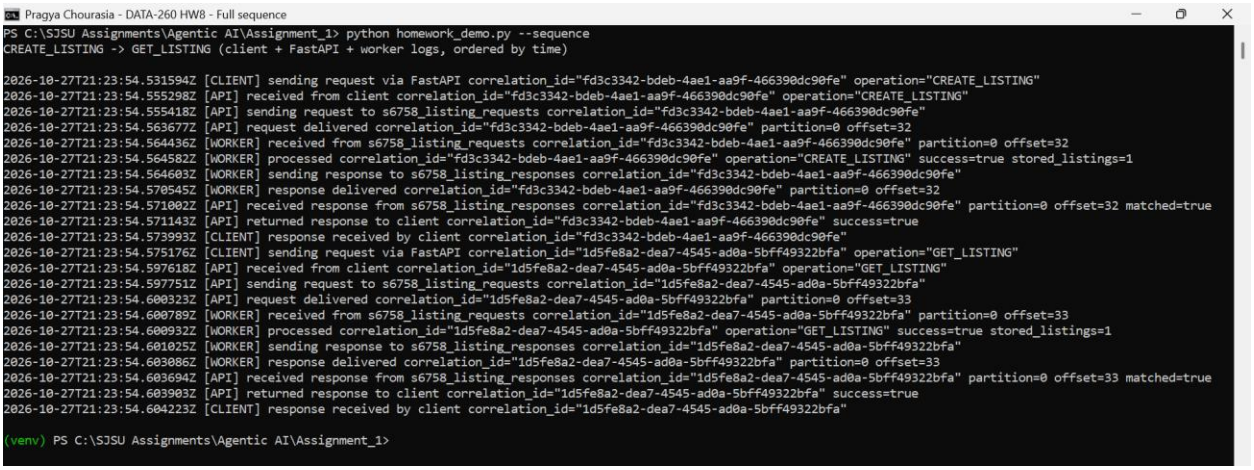


Figure 1.4: Full CREATE_LISTING -> GET_LISTING sequence in order

Part 2 - Three-Agent Kafka Pipeline

Planner, Writer and Reviewer run as separate processes. Each one reads one topic and writes the next one, so no agent calls another agent directly. All three use llama3.2:3b.

Sender	Reads	Sends to
You (send_question.py)	-	s6758_inbox (question)
Planner	s6758_inbox	s6758_tasks (plan)
Writer	s6758_tasks	s6758_drafts (draft answer)
Reviewer	s6758_drafts	s6758_final (status and answer)

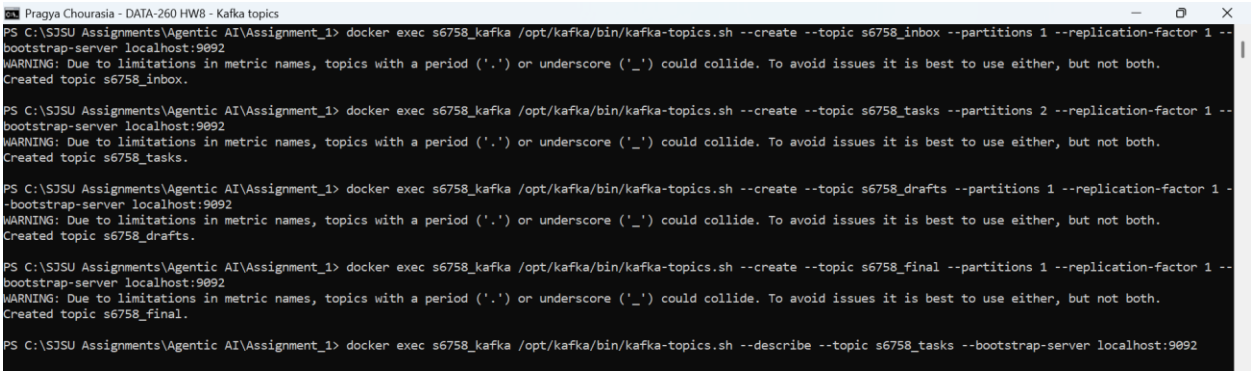
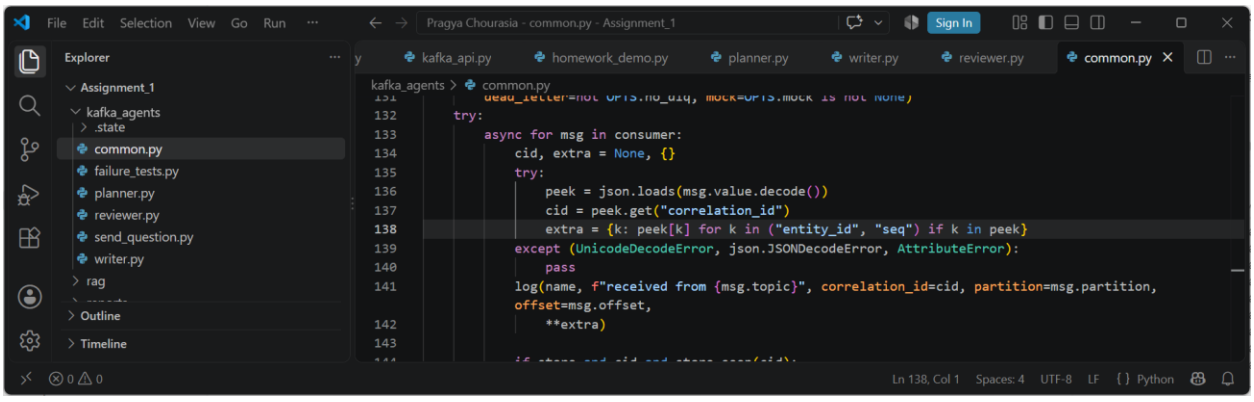


Figure 2.1: Creating the four topics with the Kafka CLI; s6758_tasks has 2 partitions

2.1 One message journey

The same correlation_id (15082799-765b-49e4-8833-e8231f4eaad2) stays on the message from s6758_inbox to s6758_final.

Shared consumer loop (read and log each message) (kafka_agents/common.py)



Planner (kafka_agents/planner.py)

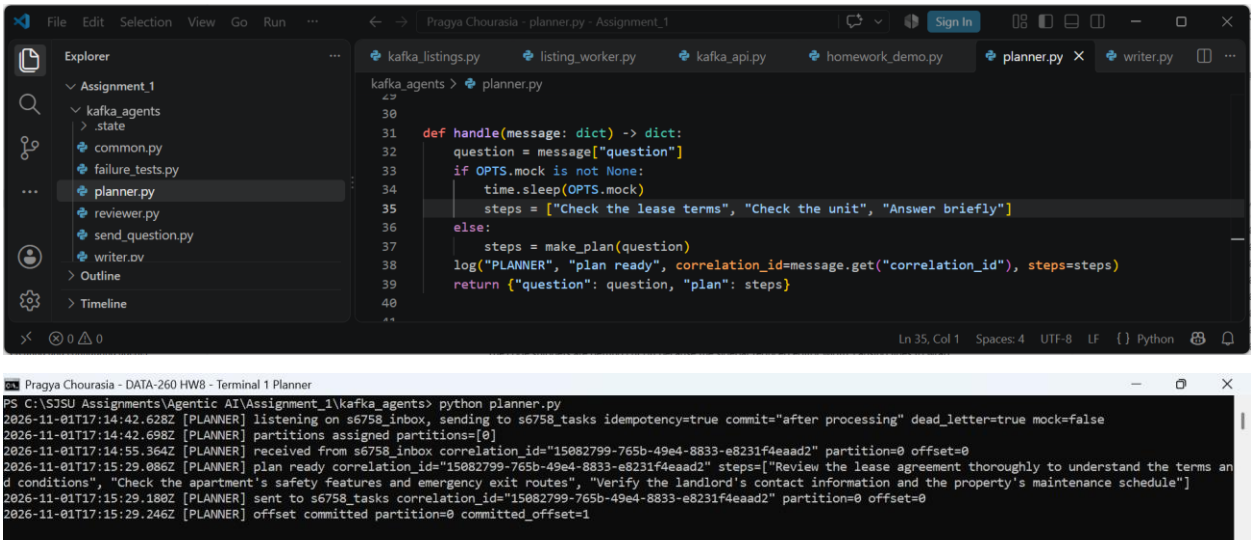


Figure 2.2: Terminal 1: Planner

Writer (kafka_agents/writer.py)

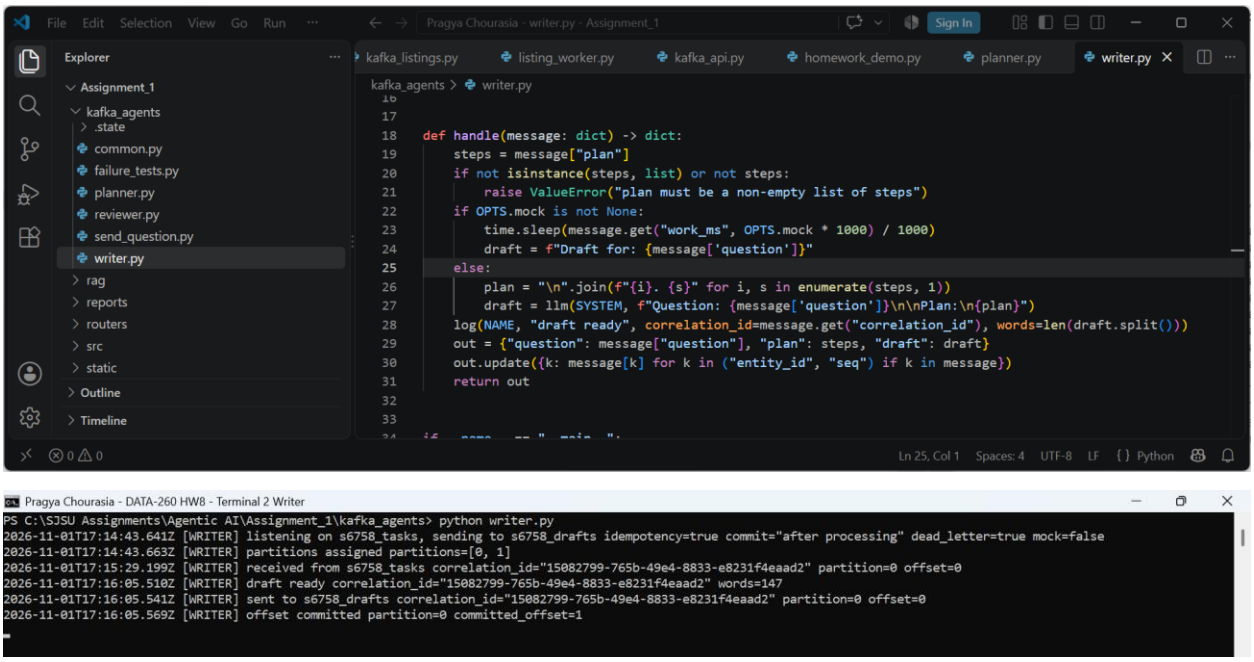
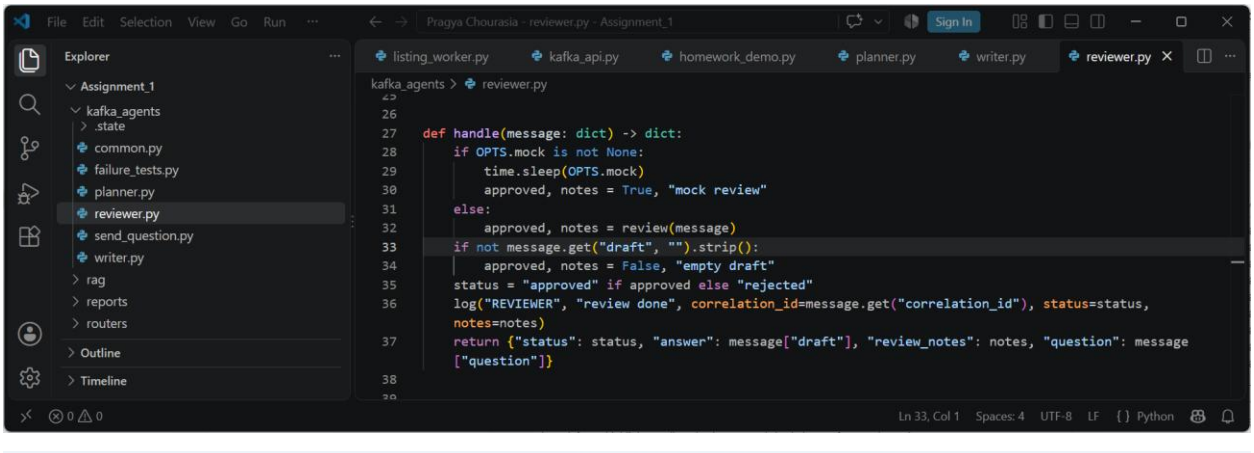


Figure 2.3: Terminal 2: Writer

Reviewer (kafka_agents/reviewer.py)



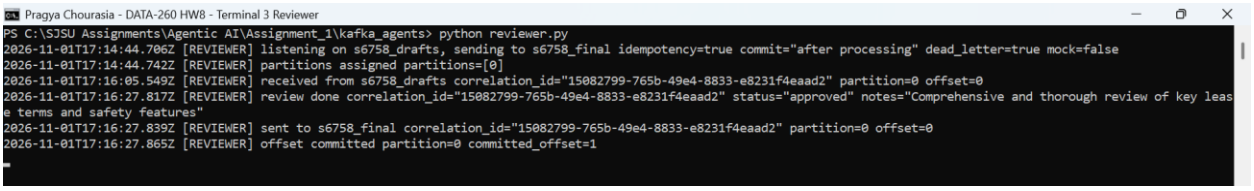


Figure 2.4: Terminal 3: Reviewer



Figure 2.5: Terminal 4: send_question.py and the final JSON from s6758_final

2.2 How Kafka helped the agents coordinate

Kafka was the only link between my agents. Each agent only needs to know its input and output topic, so I could start them in separate terminals and change one without touching the others. The agents also work at different speeds: the local model took about 20 to 35 seconds per step, and the message waits in the topic until the next agent is free. The correlation_id ties all the stages together, so send_question.py can pick out its own answer from s6758_final.

Part 3 - Failure Behavior

Each test ran twice, before and after the fix, using failure_tests.py. These tests use a --mock option on the agents (no LLM call) so they finish quickly.

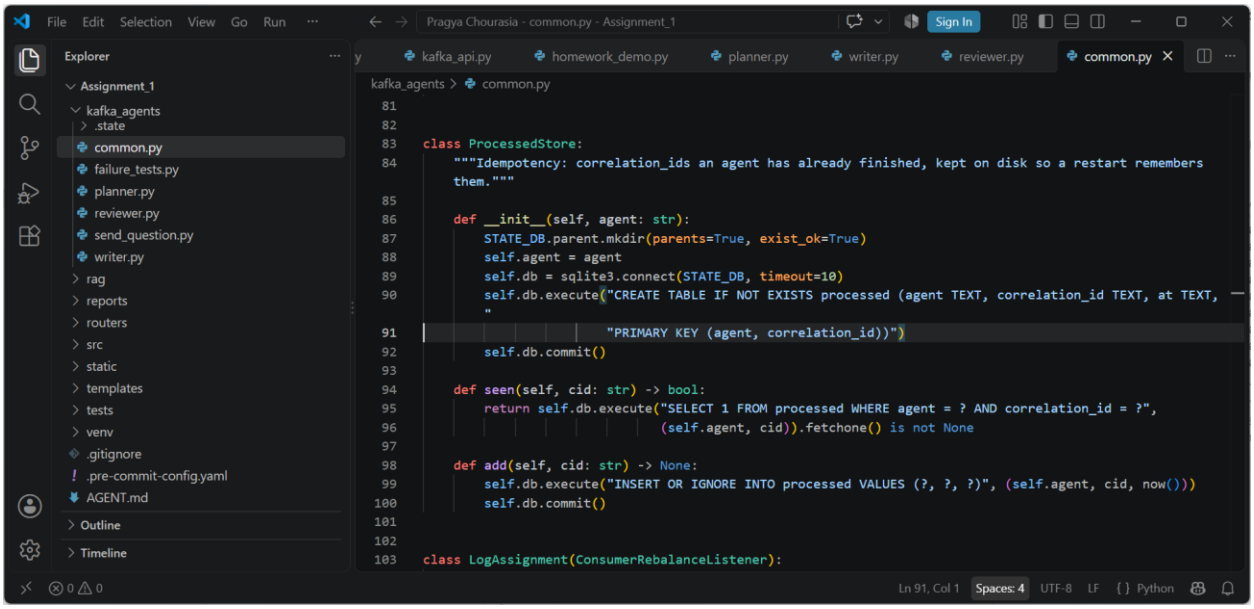
3.1 Failure table

Failure scenario	Before fix	After fix
Duplicate delivery	Replay ran every agent again: 2 Reviewer runs, 2 messages in s6758_final	Replay skipped by the Planner: 1 duplicate skipped, 1 message in s6758_final
Consumer killed mid-work	Offset already committed at the kill (1); message not redelivered, 0 drafts, so the work was lost	No offset committed at the kill; message redelivered after restart, 1 draft, offset then committed (1)
Poison message	Writer crashed 2 times on the same message; later message processed: False	3 attempts, 1 message in s6758_dead_letter, 0 crashes; later message processed: True
Two partitions/two consumers	20 processed, 2 of 3 listings split across partitions, 2 of 3 out of order (L-6758-002, L-6758-003)	20 processed, 0 of 3 listings split across partitions, 0 of 3 out of order

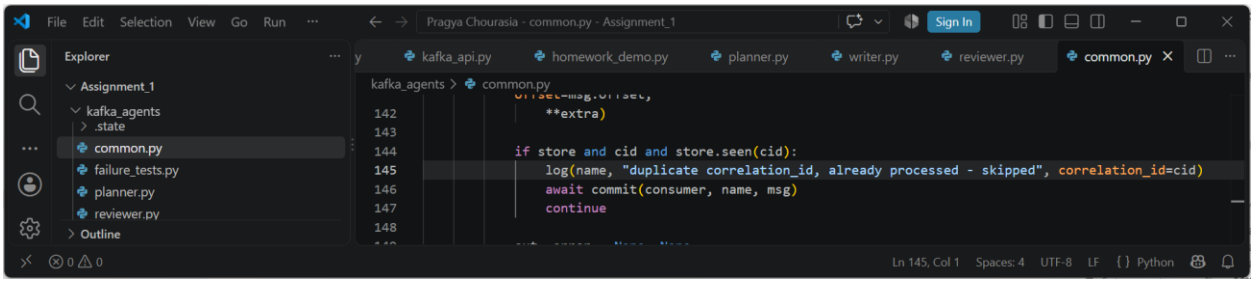
3.2 Duplicate delivery

I replayed a question with the same correlation_id. Before the fix all three agents ran again and s6758_final got a second message. Now each agent saves finished correlation_ids in SQLite and skips a repeat.

Idempotency store (kafka_agents/common.py)



Skipping a duplicate (kafka_agents/common.py)



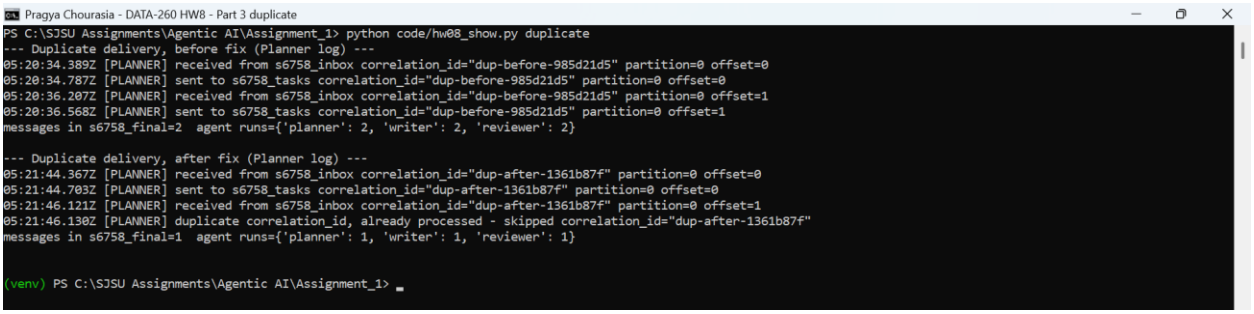
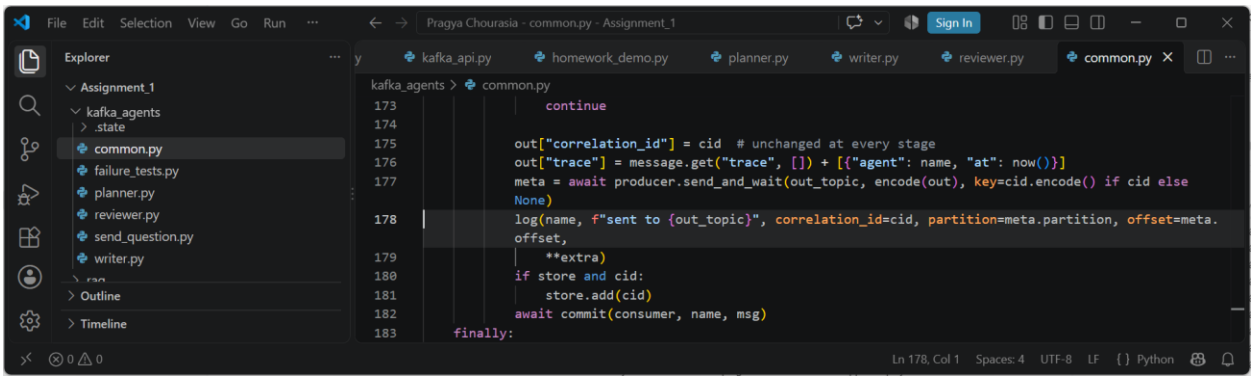


Figure 3.1: Replay before and after the fix

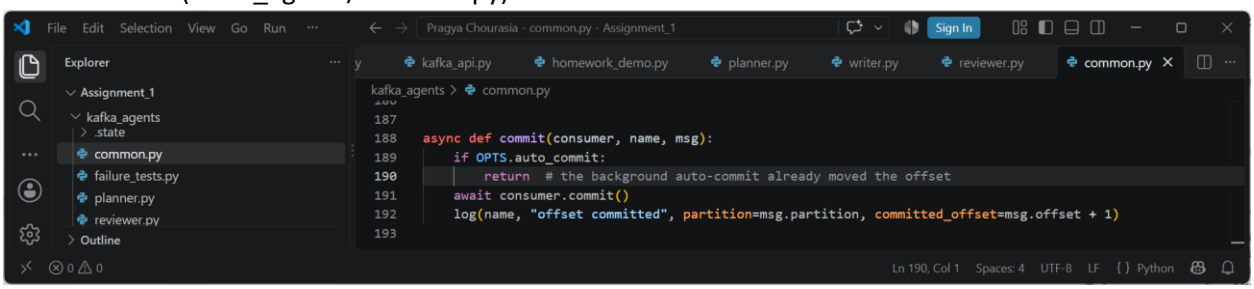
3.3 Consumer death

I killed Writer in the middle of its work and restarted it. With auto-commit the offset was already committed, so the message was never redelivered and the work was lost. Now the offset is committed only after the draft is delivered, so the restarted Writer gets the message again and finishes it.

Send, then commit (kafka_agents/common.py)



Offset commit (kafka_agents/common.py)



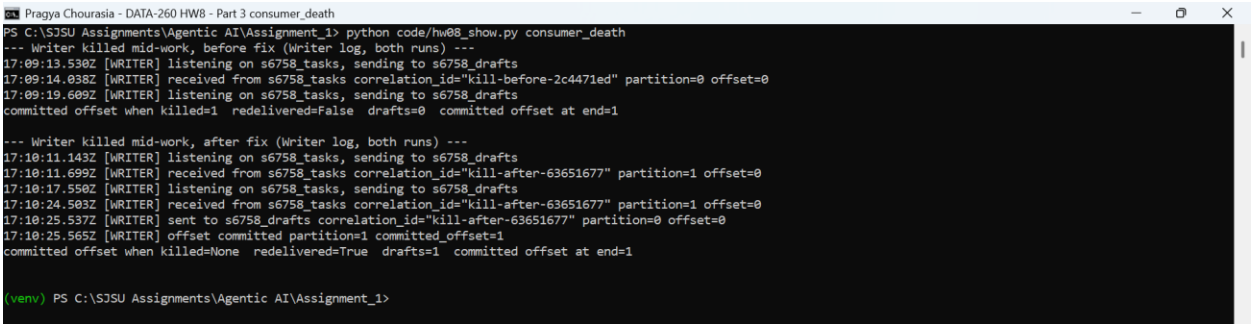
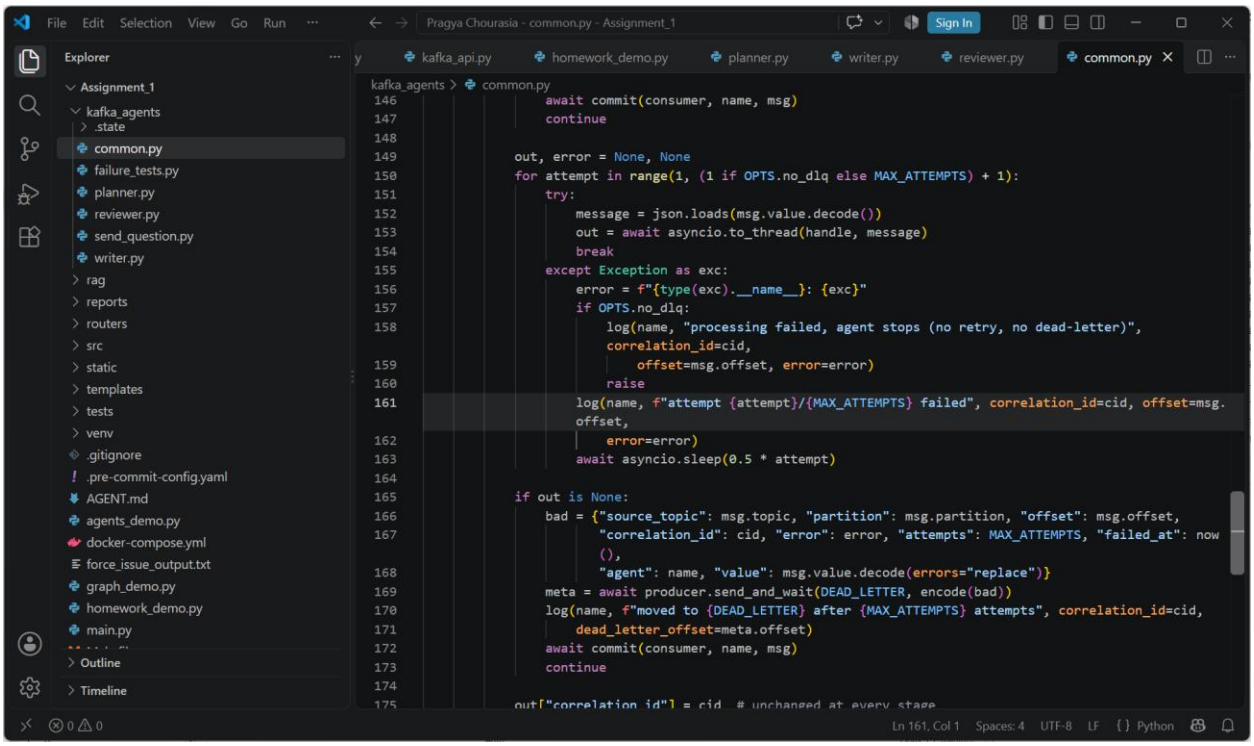


Figure 3.2: Writer killed mid-work before and after the fix

3.4 Poison message

I sent a task with a bad plan, then a good task in the same partition. Before the fix Writer crashed on the bad message every time it restarted, so the good message was stuck behind it. Now a failing message is tried 3 times and then moved to s6758_dead_letter, and the good message is processed.

Retries and dead-letter topic (kafka_agents/common.py)



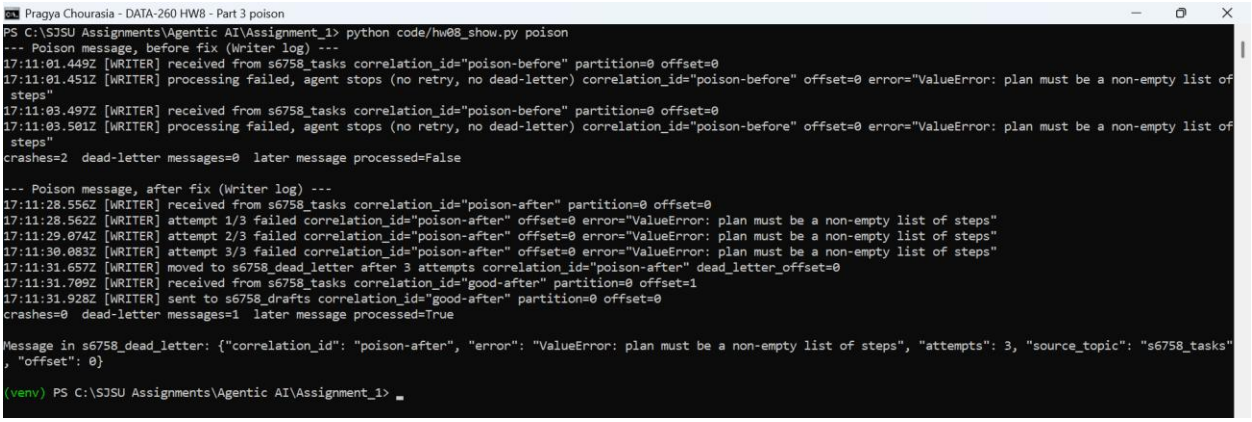


Figure 3.3: Poison message before and after the fix

3.5 Ordering

Using VERIFY_SEED 266758 I generated 20 messages about 3 listings, each with a sequence number per listing. Two Writers in one consumer group each read one partition of s6758_tasks. Without a key, two listings were split across partitions and finished out of order. With the listing id as the key, every listing stayed in one partition and kept its order. The order for each run is in raw/part3_ordering_before.csv and raw/part3_ordering_after.csv.

Partition key (kafka_agents/failure_tests.py)



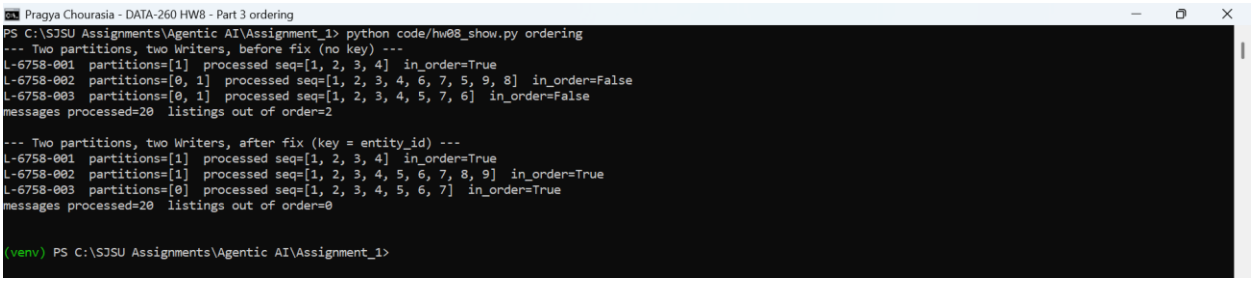


Figure 3.4: Processing order for both runs

3.6 Comparison with the HW2 graph

In HW2 my graph ran as function calls in one process, so each step waited directly on the previous step.

	Point	What I saw
Benefit 1	Independent scaling	I ran two Writers in one consumer group without changing the Planner or Reviewer
Benefit 2	Decoupled failure	When Writer was killed the message stayed in Kafka and finished after restart; in HW2 a crash stops the whole run
Responsibility 1	Duplicate delivery	The same message can arrive again, so I had to make the agents idempotent
Responsibility 2	Ordering	Order is only kept inside a partition, so I had to choose the message key

Some limits remain. If an agent dies after sending its output but before saving the correlation_id, a redelivery can still make a duplicate. The SQLite store also only works when the agents run on one machine.

AI Use

1. What I used AI for and what I did myself

I used Claude to debug problems and understand the Kafka parts. I gave it the HW8 brief and the two demo projects and used it to help write the listing worker and FastAPI wrapper, the Docker setup, the Planner, Writer and Reviewer, the failure tests, the verify script and the report layout.

I chose to continue in my rental listings repo, ran the system on my laptop (Docker and Ollama), checked the outputs and screenshots against the brief, and did the final Git push and tag myself.

2. Something the AI got wrong

- The first merged Part 1 log was out of order. The worker's "received" line appeared before the API's "sent" line, because the API only logged after Kafka acknowledged the send, and lines with the same millisecond were sorted by text.
- The first Part 2 screenshots were wrong. The Planner image showed the send_question window because the script captured the screen without bringing the Planner window to the front.

3. How the problems were found

- Reading the sequence log line by line: a message cannot be received before it is sent.
- Opening each screenshot and comparing the window title with the expected terminal.

4. What was changed

- The API and worker now log right before sending, timestamps use microseconds, and the merge sorts only on the timestamp. The sequence now reads send, receive, process, reply, client response.
- The screenshot script minimizes and restores each window before capturing it, and each window is maximized before it prints anything, so the output starts at the top.